

Practical Experiment: Compare the response speed of open-source Large Language Models (LLMs) to find the fastest model for real-time applications

Meeting 7 PIP Package to Web

Presented by: Mariyan Vasilev Apostolov
Date: 03th of September 2026
Version: 1.0

Agenda

- FastAPI + React
- Introduction to FastAPI
- Introduction to React
- Architectural Overview – Three-Tier Design
- Authentication & Best Practices



FastAPI + React

The Perfect Modern Stack

- FastAPI: High-performance, async Python framework
- React: Component-based UI library
- Clean separation: API backend + SPA frontend
- Auto docs: Swagger UI out of the box
- Type safety: Python types + TypeScript
- Industry adoption: Used by Netflix, Uber, Microsoft



Introduction to FastAPI

FastAPI is a modern, high-performance Python web framework designed for building APIs with standard Python type hints. Its architectural foundation rests on two key libraries: Starlette for asynchronous web handling and Pydantic for data validation and serialization.

Key Characteristics:

- **Type-Driven Development:** FastAPI leverages Python's type annotation system to automatically validate request and response data, generate interactive documentation and provide editor support. This reduces boilerplate and minimizes runtime errors.
- **Asynchronous Concurrency:** Built on ASGI (Asynchronous Server Gateway Interface), FastAPI supports `async/await` syntax natively, enabling efficient handling of I/O-bound operations such as database queries or external API calls.
- **Automatic Documentation:** The framework generates OpenAPI-compliant documentation accessible through Swagger UI and ReDoc interfaces without additional configuration.

In the context of LLM testing, FastAPI serves as an ideal orchestration layer. It can manage concurrent test requests, validate incoming parameters, and coordinate communication with the Ollama service.

5 Introduction to React

React is a JavaScript library for building user interfaces through a declarative, component-based paradigm. React has become the dominant approach for constructing interactive web applications.

Fundamental Principles:

- **Declarative Rendering:** React efficiently updates the DOM when data changes. This abstraction eliminates manual DOM manipulation and reduces error surfaces.
- **Component Composition:** Applications are constructed from encapsulated, reusable components that manage their own state and can be composed into complex interfaces. This modularity supports maintainability and testing.
- **Unidirectional Data Flow:** Props flow downward from parent to child components, while callbacks enable child-to-parent communication. This predictable data flow simplifies reasoning about application state.

React development employs Vite as the build tool, which provides faster development server startup and hot module replacement compared to earlier toolchains.

Architectural Overview – Three-Tier Design

The proposed system follows a three-tier architecture with clear separation of concerns:

Tier 1: Ollama Inference Service

Operating on a dedicated port (11434), Ollama manages model loading and inference. It exposes a REST API that FastAPI can call using the Ollama Python client. This isolation ensures that model execution does not interfere with web request handling.

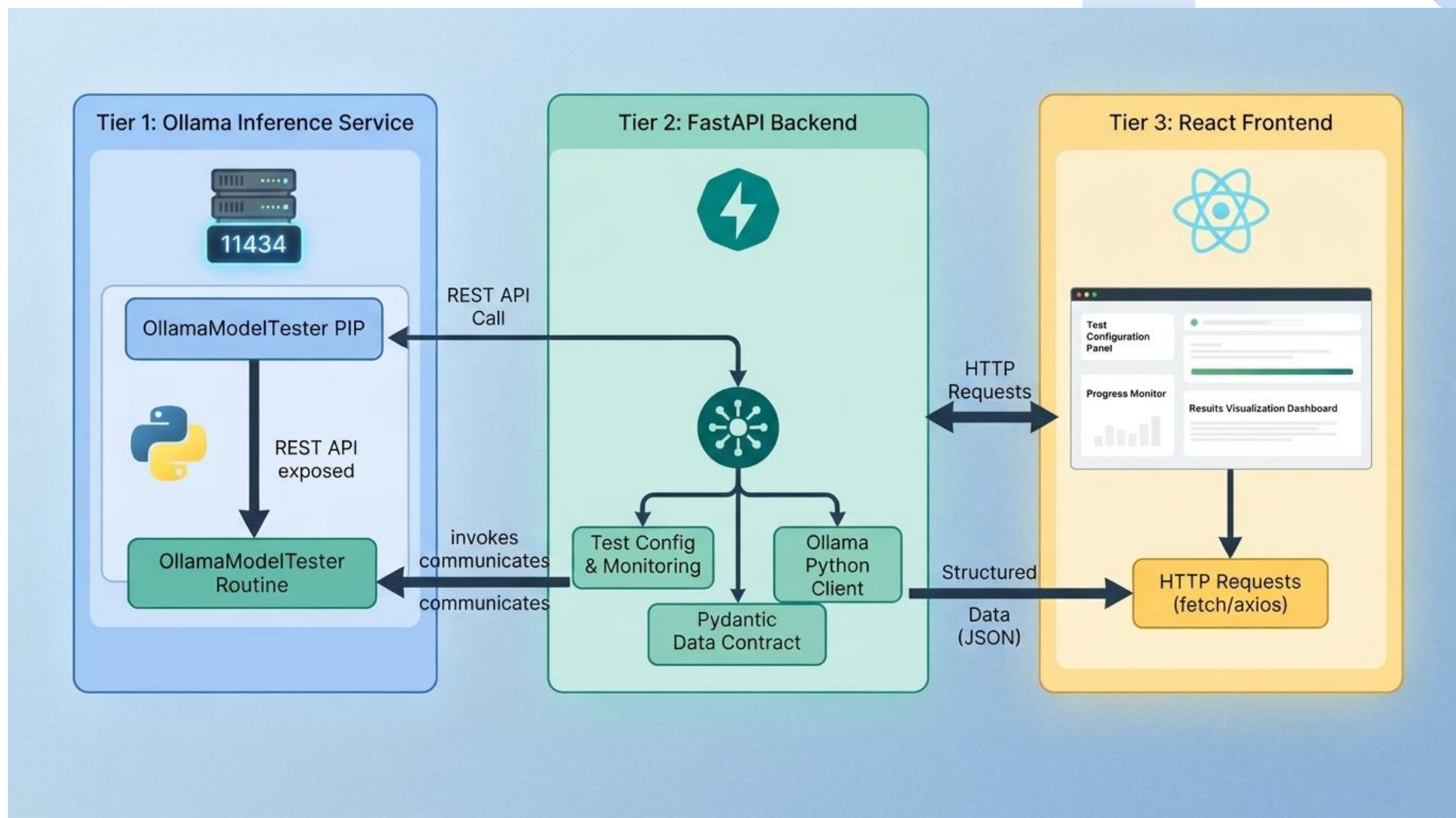
Tier 2: FastAPI Backend

The backend serves as the orchestration layer. It receives HTTP requests from React, invokes OllamaModelTester routines, communicates with Ollama, and returns structured results. FastAPI's Pydantic models define the contract between frontend and backend, ensuring type safety across the boundary.

Tier 3: React Frontend

The presentation layer provides an interactive interface for configuring tests, monitoring progress, and visualizing results. React communicates with FastAPI through HTTP requests, using fetch or axios libraries.

Architectural Overview – Three-Tier Design



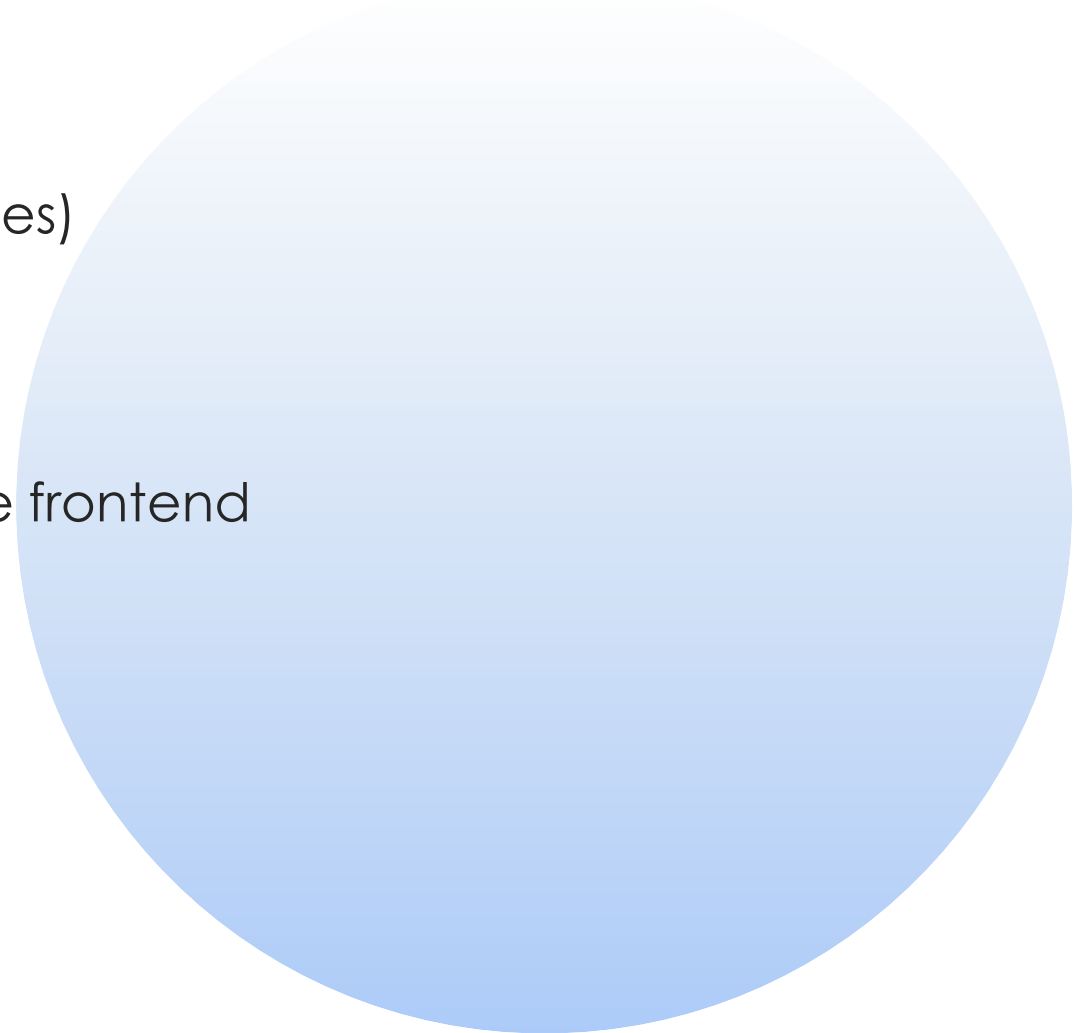
Authentication & Best Practices

Security

- JWT tokens (python-jose, passlib)
- OAuth2 password flow in FastAPI
- Store tokens securely (HTTP-only cookies)

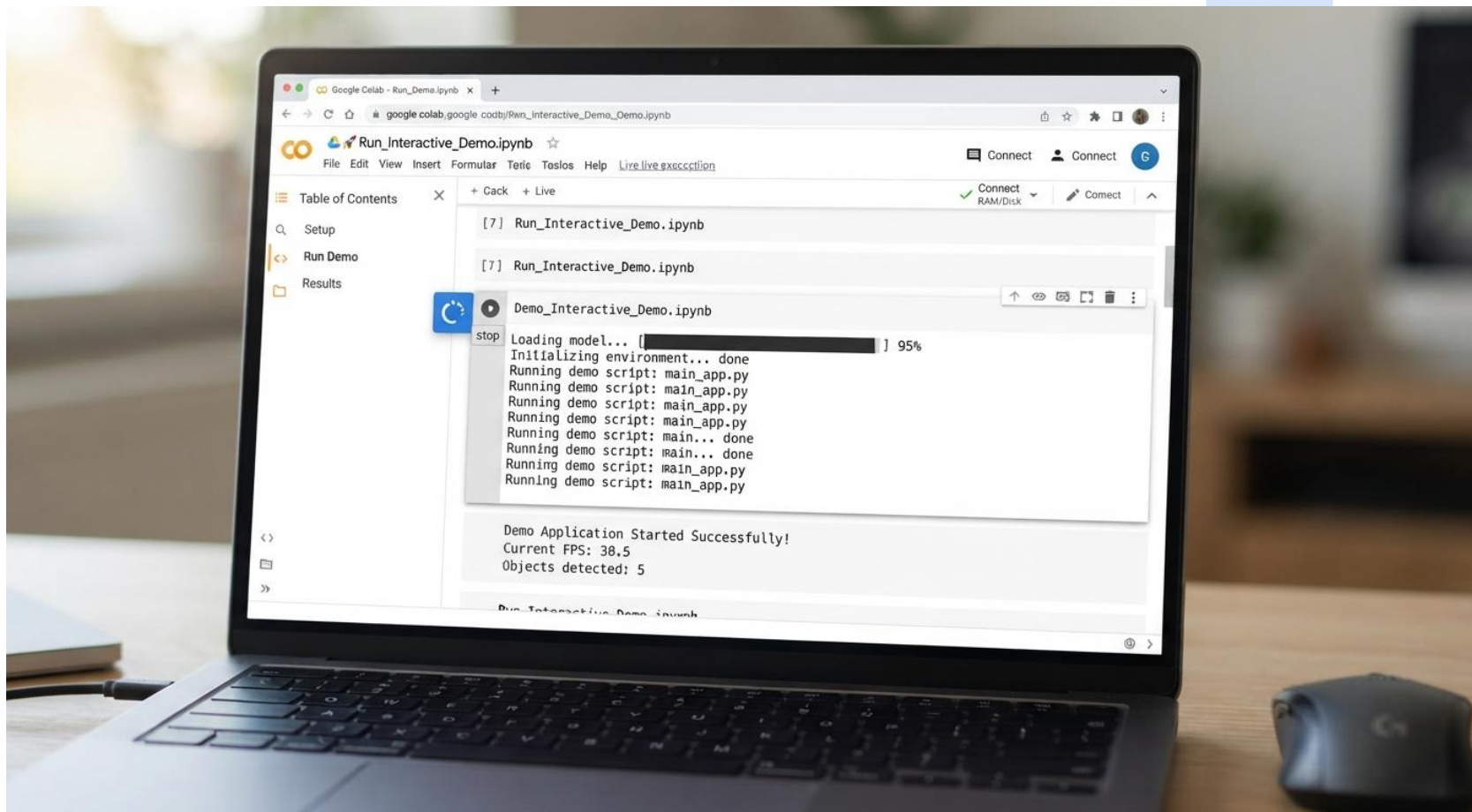
Best Practices

- Pydantic schemas for validation
- Separate routers by domain
- Native JavaScript or TypeScript on the frontend
- Async DB calls
- Environment-based configs



9

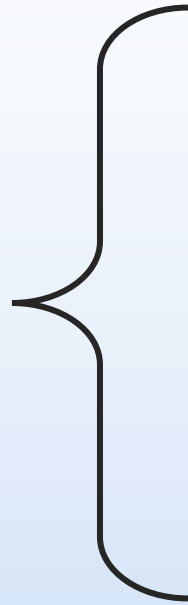
LIVE DEMO



References

- FastAPI Documentation – <https://fastapi.tiangolo.com/>
- FastAPI Documentation – <https://fastapi-tutorial.readthedocs.io/en/latest/#tutorial>
- React with Vite Documentation – <https://react.dev/learn/build-a-react-app-from-scratch>
- Vite DEV – <https://vite.dev/guide/>

Thank you



Mariyan Vasilev Apostolov

mariyan.apostolov89@gmail.com